

III. FILES

FILES:

A file can be defined as a location for storing related data. A file holds a specific name for itself. A file can be a sequence of bits, bytes, lines or records depending on the application used to create it.

Example:

sample.py, student.txt –Here sample, student are the file names with file extensions

Types of files:

Files can be categorized as follows:

Text files:

A text file is a file containing characters structured as individual lines of text. In addition to printable characters, text files also contain the nonprinting newline character `\n` to denote the end of each text line.

Binary files:

Binary files can contain various types of data, such as numerical values. These are not structured as lines of text. Such files can only be read and written via a computer program.

S.no	Type	Application
1.	Text	Notepad
2.	Audio	MediaPlayer
3.	Video	Mediaplayer
4.	Presentation	Mediaplayer
5.	Doc	Word

ifferent Modes to Open a File in Python

Mode	Description
R	Open a file for reading. (default)
W	Open a file for writing. Creates a new file if it does not exist or truncates the file if it exists.
X	Open a file for exclusive creation. If the file already exists, the operation fails.
A	Open a file for appending at the end of the file without truncating it. Creates a new file if it does not exist.
T	Open in text mode. (default)
B	Open in binary mode.
+	Open a file for updating (reading and writing)

FILE OPERATIONS:

Files in python can be manipulated via the following file operations:

1. Opening a file
2. Reading from a file
3. Writing to a file
4. Appending a file
5. Closing a file

Opening a file

All files must be opened first before they can be used. In python, when a file is opened, a file object is created that provides methods for accessing the file. In python there is a built in function `open()` to open a file.

Syntax:

```
file_object=open(filename,accessmode)
```

where file_object is used as a handle to the file

file_name-contains a string type value containing the name of the file which we want to access

access_mode-specifies the mode in which we want to open the file

```
file = open("File1.txt", "rb")  
print(file)
```

OUTPUT

```
<open file 'File1.txt', mode 'rb' at 0x02A850D0>
```

#Example for opening a file

```
fp1=open("sample.txt","w")  
print(fp1)  
fp=open("sample.txt","r")  
print(fp)
```

Output:

```
<_io.TextIOWrapper name='sample.txt' mode='w' encoding='cp1252'>  
<_io.TextIOWrapper name='sample.txt' mode='r' encoding='cp1252'>
```

#Example for opening a file when the file does not exists

```
fp1=open("sample1.txt","r")  
print(fp1)
```

Output:

```
fp1=open("sample1.txt","r")  
FileNotFoundError: [Errno 2] No such file or directory: 'sample1.txt'
```

PYTHON FILE OPENING MODES:

Modes	Description
r	Opens a file for reading only. The file pointer is placed at the beginning of the file. This is the default mode.
rb	Opens a file for reading only in binary format. The file pointer is placed at the beginning of the file. This is the default mode.
r+	Opens a file for both reading and writing. The file pointer is placed at the beginning of the file.
rb+	Opens a file for both reading and writing in binary format. The file pointer is placed at the beginning of the file.
w	Opens a file for writing only. Overwrites the file if the file exists. If the file does not exist, creates a new file for writing.
wb	Opens a file for writing only in binary format. Overwrites the file if the file exists. If the file does not exist, creates a new file for writing.
w+	Opens a file for both writing and reading. Overwrites the existing file if the file exists. If the file does not exist, creates a new file for reading and writing.
wb+	Opens a file for both writing and reading in binary format. Overwrites the existing file if the file exists. If the file does not exist, creates a new file for reading and writing.
a	Opens a file for appending. The file pointer is at the end of the file if the file exists. That is, the file is in the append mode. If the file does not exist, it creates a new file for writing.

ab	Opens a file for appending in binary format. The file pointer is at the end of the file if the file exists. That is, the file is in the append mode. If the file does not exist, it creates a new file for writing.
a+	Opens a file for both appending and reading. The file pointer is at the end of the file if the file exists. The file opens in the append mode. If the file does not exist, it creates a new file for reading and writing.
ab+	Opens a file for both appending and reading in binary format. The file pointer is at the end of the file if the file exists. The file opens in the append mode. If the file does not exist, it creates a new file for reading and writing.

Note: When opening a file if the access mode is not specified, the file is opened in read mode

File object attributes:

Once a file is opened and a file object is created, various pieces of information about the file can be gathered by using some predefined attributes. There are basically four types of attributes as shown below:

Attribute	Description
file.closed	Returns true if file is closed, false otherwise.
file.mode	Returns access mode with which file was opened.
file.name	Returns name of the file.
file.softspace	Returns false if space explicitly required with print, true otherwise.

```
file = open("File1.txt", "wb")
print("Name of the file: ", file.name)
print("File is closed.", file.closed)

print("File has been opened in ", file.mode, "mode")
```

OUTPUT

```
Name of the file: File1.txt
File is closed. False
File has been opened in wb mode
```

#File object attributes

```
fp=open("sample.txt","r")
print("Name of the file",fp.name)
print("closed or not",fp.closed)
print("opening mode",fp.mode)
print("File object",fp)
fp.close()
print("Name of the file",fp.name)
print("closed or not",fp.closed)
print("opening mode",fp.mode)
print("File object",fp)
```

Output:

```
Name of the file sample.txt closed ornot False opening mode r
File object <_io.TextIOWrapper name='sample.txt' mode='r' encoding='cp1252'>
Name of the file sample.txt closed ornot True
opening mode r
File object <_io.TextIOWrapper name='sample.txt' mode='r' encoding='cp1252'>
```

Reading from a file

In order to read from a file, the file must be opened in r mode. A file can be read in four different ways:

- read(size)
- readline()

- readlines()
- loop

read() method:

The read() enables reading the contents of an opened file.

Syntax:

```
fileobject.read()  
fileobject.read(size)
```

where size argument is optional. When the size is not specified, the entire file is read. When the size is specified, the size number of characters is read. When an empty file is read, no error is thrown but an empty string is returned.

readline() method:

This method is used for reading a file line by line. This method reads a file till the newline character including the newline character.

Syntax:

```
fileobject.readline() fileobject.readline(size)
```

where size is optional. When size is specified the size number of characters are read, otherwise it is similar to read()

readlines() method:

The readlines() method returns a list of lines of the entire file.

Syntax:

```
fileobject.readlines()  
fileobject.readlines(size)
```

When size is not specified, the entire file is read and each line is added as an element in a list.

Using loop:

A file can be read line by line using for loop

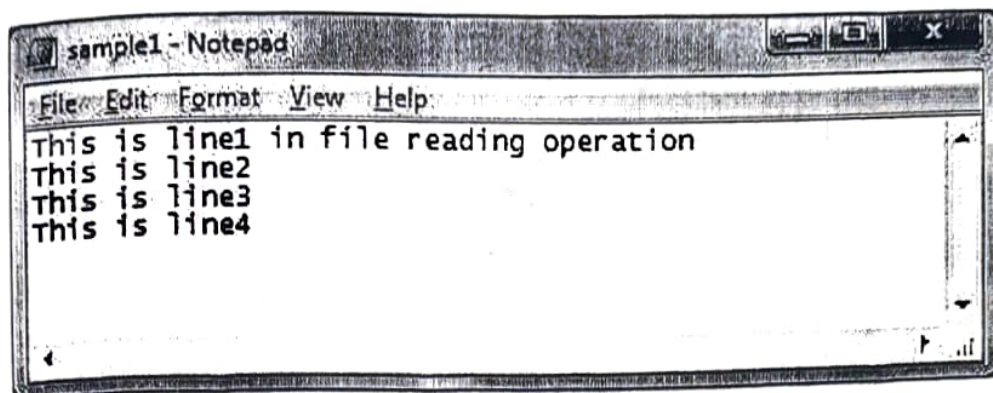
Syntax:

```
for line in <filevariablename>:
```

```
    statements for file operations
```

Example for reading a file using various methods:

Let sample1.txt be the file



#Reading a file using read() method

```
fp=open("sample1.txt","r")  
print("Read a file using read()")  
print(fp.read())  
fp.close()
```

Output:

```
Read a file using read()  
This is line1 in file readingoperation  
This is line2  
This is line3  
This is line4
```

#Reading a file using read(size) method

```
fp=open("sample1.txt","r")  
print(fp.read(10))  
fp.close()
```

Output:

Read a file using read(size)

This is li

#Reading a file using readline() method

```
fp=open("sample1.txt","r")
print("Read a file using readline()")
print(fp.readline())
fp.close()
```

Output:

Read a file using readline()

This is line1 in file reading operation

#Reading a file using readlines() methods

```
fp=open("sample1.txt","r")
print("Read a file using readlines()")
print(fp.readlines())
fp.close()
```

Output:

Read a file using readlines()

['This is line1 in file reading operation\n', 'This is line2\n', 'This is line3\n', 'This isline4']

#Reading a file using loop `fp=open("sample1.txt","r")`

for line in fp:

```
    print("The contents of the file are")
print(line)
fp.close()
```

Output:

The contents of the file are
This is line1 in file reading operation
The contents of the file are
This is line2
The contents of the file are
This is line3
The contents of the file are
This is line4

Writing to a file

In order to write into a file, the file has to be opened in "w" mode or "a" mode or any other writing mode. The "w" mode overwrites the file if the file already exists. If the file does not exist, a new file is opened for writing.

Syntax:

```
fileobject=open("filename","w")
```

The write operation can be done using the following ways:

- write()
- writelines()

write() method:

The write method puts data into the file.
The write() method writes any string to an open file.
The write() method does not add a newline character ('\n') to the end of the string

Syntax:

```
fileobject.write(string)
```

Where string is the content that is written to the file

writelines()

The writelines method puts multiple data into the file. The writelines() method writes any string to an open file.

Syntax : fileobject.writelines(string)

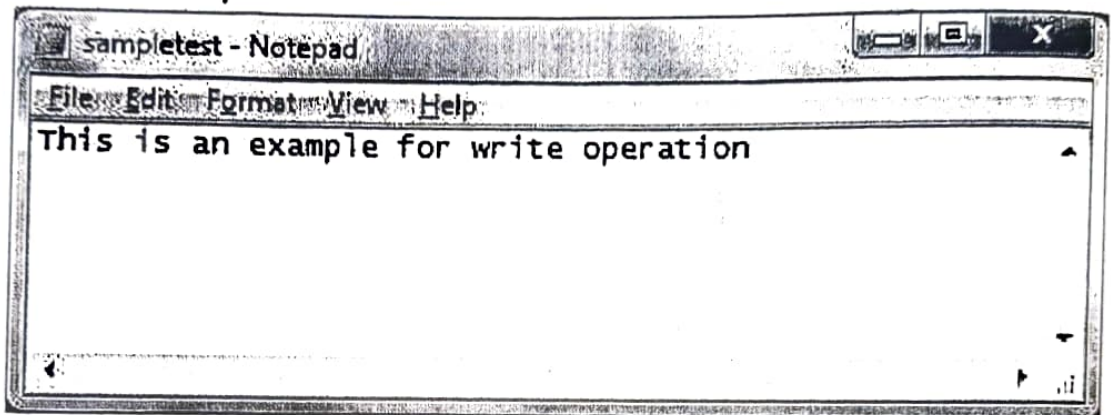
Example for writing a file when the file already exists

#Writing to a file when the file already exists `fp=open("sampletest.txt","w")`

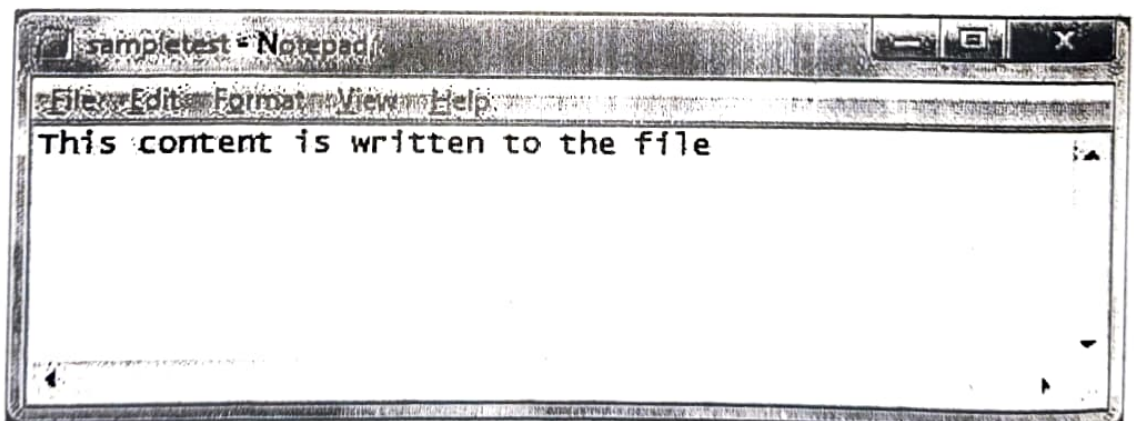
```
fp.write("This content is written to the file")  
fp.close()
```

Output:

Before write operation

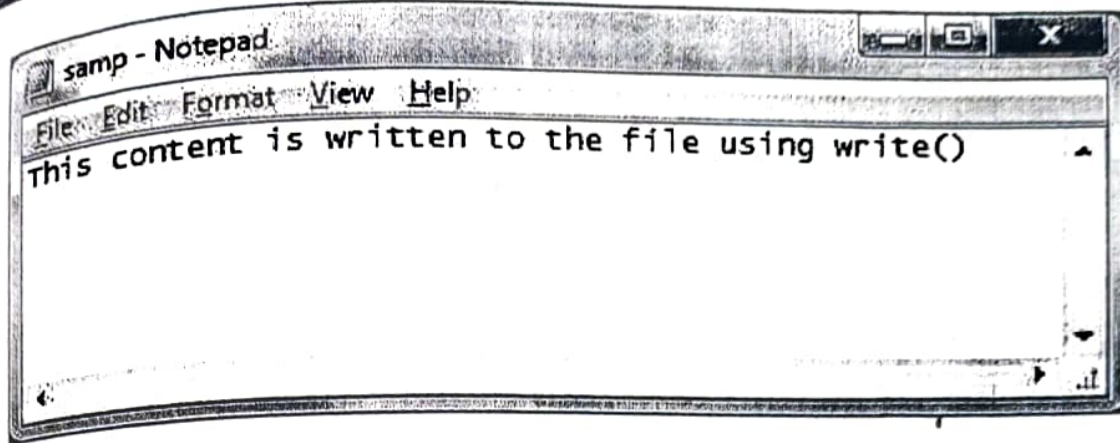


After write operation

**Example:**

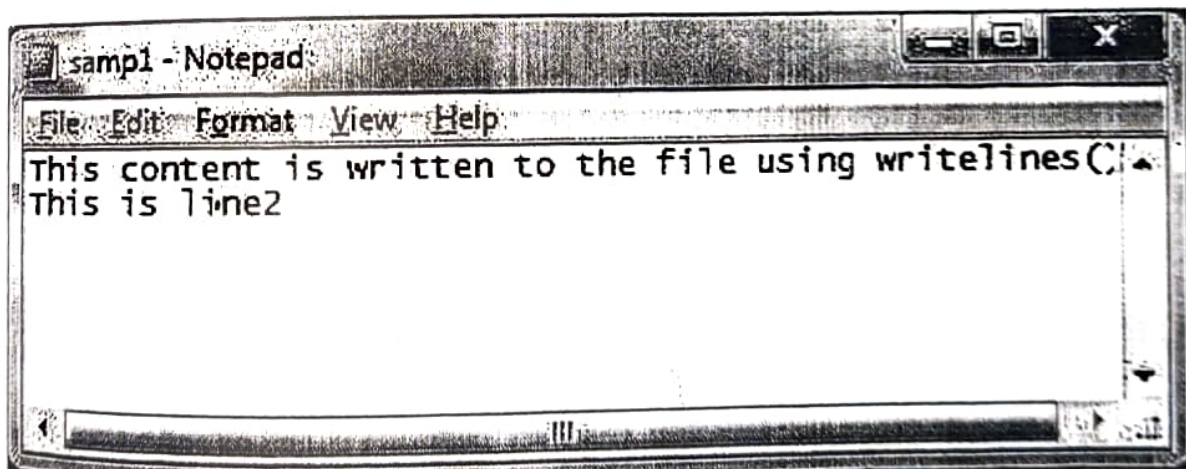
#Writing to a file using `write()`

```
fp=open("samp.txt","w")  
fp.write("This content is written to the file using write()")  
fp.close()
```



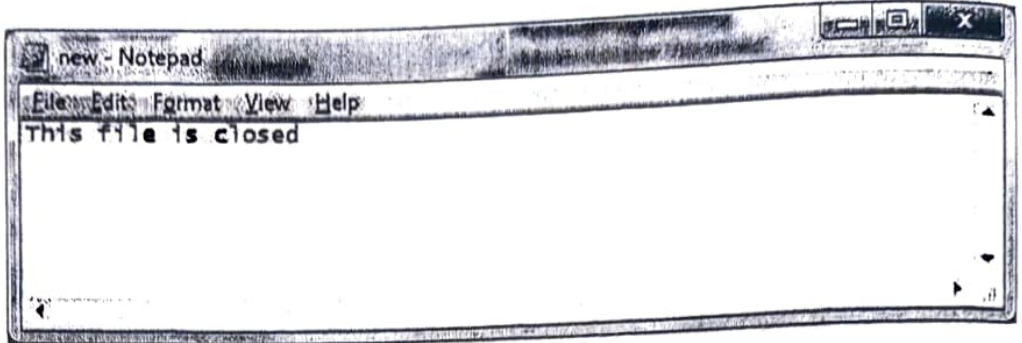
Example:
#Writing to a file using writelines()

```
fp=open("samp1.txt","w")  
fp.writelines("This content is written to the file using writelines()\nThis is line2")  
fp.close()
```



Closing a file:

When the operations that are to be performed on an opened file are finished, the file has to be closed in order to release the resources. Proper closing of file frees up the resources held with the file. The closing of file is done with the built-in function `close()`

Syntax:`fileobject.close()`**Example:****#Closing a file**

```
fp=open("new.txt","r")
print(fp.read())
fp.close()
```

Output:

This file is closed

Closing a file using with:

A file can also be closed using with the statement as follows:

Syntax:

`with open("filename",accessmode) as fileobject:statements for file operations`

#Closing a file using with

```
with open("new.txt","r") as fp:print(fp.read())
fp.close()
```

Output:

This file is closed